Interpretable Differences in Reasoning: Crosscoder Diffing Between Fine-Tuned and Base Qwen-2.5-0.5B-Instruct

Tyler Crosse Juan Molina Carlos Iraheta
Georgia Institute of Technology
{tcrosse3, jmolina33, ciraheta3}@gatech.edu

Abstract

Improving the reasoning capabilities of Large Language Models (LLMs) often involves fine-tuning on specialized datasets, yet the internal mechanistic changes underlying these improvements remain poorly understood. This work investigates these changes by fine-tuning a small LLM, Qwen2.5-0.5B-Instruct, on the s1k reasoning dataset derived from state-of-the-art models. We confirm that fine-tuning enhances reasoning performance, particularly the model's ability to scale its accuracy with increased test-time steps on math benchmarks like gsm8k and MATH 500. To probe the internal modifications, we used a Crosscoder, a specialized sparse autoencoder variant. to compare latent activations between the base and fine-tuned models at a mid-network layer. While the Crosscoder successfully reconstructs activations with high fidelity (achieving over 93% Cross-Entropy Loss Recovery), analysis of its learned features reveals that the representational geometry at this layer remains surprisingly conserved. Despite performance gains, the majority of features identified by the Crosscoder are shared and highly aligned between the base and fine-tuned models. This suggests that the functional improvements in reasoning may stem from subtle representational shifts, changes in later layers, or modifications in how existing features are utilized. This highlights the ongoing challenge of pinpointing the precise internal adaptations responsible for complex cognitive enhancements in LLMs.

1. Introduction

In recent years, Large Language Models (LLMs), largely based on the Transformer architecture [25], have demonstrated remarkable generalization capabilities across a wide array of applications [6]. A significant research thrust focuses on enhancing their capacity for complex reasoning [11], often inspired by human problem-solving strategies like chain-of-thought (CoT) [27]. State-of-the-art reasoning models, such as OpenAI's o3 [20, 21] and DeepSeek's R1 [7], excel on verifiable domains like mathematics and coding. These models often exhibit extended inference computations ("thinking time") and generate explicit reasoning steps, sometimes backtracking to correct errors [13].

1.1. Limitations of Current Approaches

Despite these impressive performance gains, achieved through methods like scaling or specialized fine-tuning, *how* these models internally adapt their computational mechanisms to perform step-by-step reasoning remains largely opaque. The models are often treated as black boxes; we observe improved outputs, but lack a fundamental understanding of the underlying changes. In particular, how a model "decides" to backtrack or determines it has reached a correct intermediate step is a poorly understood aspect of an active research area [27, 26, 23, 28].

Mechanistic Interpretability (MI) is a subfield of deep learning dedicated to reverse-engineering neural networks to understand their internal workings [23]. Landmark MI research on LLMs has focused on extracting interpretable features and understanding phenomena like superposition, where individual neurons encode multiple concepts [8, 24, 19]. A common tool in MI is the Sparse Autoencoder (SAE), designed to disentangle learned representations within a specific layer's activations into a sparser, potentially more interpretable feature space [8, 24]. More recently, Crosscoders have emerged as a technique specifically tailored to interpret and compare latent representations across multiple layers or even between different model states, such as before and after fine-tuning [14, 1, 15, 5].

A recent and relevant example of enhancing reasoning through fine-tuning is presented in "s1: Simple test-time scaling" [16]. The authors aimed to find a simple yet effective approach to achieve strong reasoning performance via test-time scaling. They curated the s1K dataset [4], comprising 1,000 reasoning traces from DeepSeek R1, and fine-tuned Qwen2.5-32B-Instruct on this data. Their resulting model, s1-32B, demonstrated significant improvements,

notably exceeding o1-preview on competition math benchmarks. Crucially, the base model, the fine-tuned model derived from it, the s1K dataset, and the training code are all publicly available, providing an excellent testbed for investigating the internal changes induced by reasoning-focused fine-tuning.

1.2. Objective

We wanted to understand exactly how teaching a small AI language model to 'think step-by-step' changes its internal calculations. Leveraging this setup, our objective is to identify and interpret the specific internal representational changes that occur when a language model is fine-tuned for explicit step-by-step reasoning. Concretely, we fine-tune the smaller Qwen2.5-0.5B-Instruct model [22] on the s1K reasoning dataset [16, 4]. We then train a Crosscoder [14, 1] to differentiate the internal activations between the base Qwen2.5-0.5B model and its s1K-finetuned counterpart. Our goal is to pinpoint interpretable features or neuronal activation patterns that emerge or change due to the fine-tuning process and help explain the improved reasoning capabilities of the specialized model.

1.3. Motivation

This approach is motivated by the limitations of current practices. While scaling models or fine-tuning on specialized data demonstrably improves reasoning performance [21, 20, 7, 16], these methods offer little insight into the internal mechanisms. Standard analysis often relies on evaluating final task performance or employs interpretability tools like conventional SAEs [9] that typically analyze representations within a single model or layer state. These approaches are less suited for precisely identifying the *differential changes* across multiple layers that are specifically induced by the fine-tuning process and contribute functionally to the enhanced reasoning ability. Our work directly targets this gap by using Crosscoders to isolate the modifications linked to reasoning improvement.

Understanding these internal mechanisms is crucial. It moves beyond observing correlations to potentially explaining how reasoning emerges, enabling more targeted and efficient methods to enhance capabilities, especially in smaller models. Furthermore, mechanistic transparency is vital for AI safety and reliability, allowing better prediction of failure modes and fostering trust [23, 8, 24]. Success here would provide concrete evidence linking fine-tuning to specific representational changes and demonstrate a methodology for finding them [14].

1.4. Data

For finetuning, we used the dataset called s1K-1.1 from the s1: Simpletest-timescaling paper [16, 4]. It is a filtered down dataset of 1000 samples selected from a collective total of 59,029 questions. These questions were selected from NuminaMATH, AIME, OlympicArena, OmniMath, AGIEval, and two custom datasets called s1-prob and s1-teasers. Each question in this collective new dataset includes the reasoning trace and response from Google Gemini and Deepseek R1. Originally, the researchers only used Gemini traces but found that including Deepseek traces greatly improved performance. Each sample had the format of solution, question, category, source, metadata, gemini_thinking_trajectory, gemini_attempt, deepseek_thinking_trajectory, deepseek_attempt.

2. Approach

2.1. Fine Tuning

The goal of finetuning was to improve the model's ability to reason without training the model from scratch. Using information distilled from larger models, we were able to improve the results from a lower parameter model on local hardware. The hardware used was a Nvidia 5070 ti with 16 GB of VRAM, much smaller than the cluster of 16 H100 GPUs used in the original paper. Fine-tuning was performed on two specific models, Llama-3.2-1B and Qwen2.5-0.5B-Instruct [22]. The training portion was largely based on the GitHub that was provided with the s1 paper[17, 16]. The primary goal was to achieve quantifiable results on completely local hardware, so both the models and dataset were stored locally to simulate an entirely closed environment.

2.2. Evaluation

Evaluating reasoning in an LLM model can be difficult, since just defining what constitutes reasoning can be debatable. However, most texts agree that a reasoning model should be one that improves its conclusions when given more 'time' (generating tokens, in this case) to give a definite answer[16]. Given this, many benchmark tests consist of asking complex questions (on topics such as math, science, or common sense), providing a space for the LLM to provide its reasoning, and then a space to provide a final answer. The given final answer is then compared to the actual answer.[22]

In this text, we wish to test the reasoning power of our model. For this reason, simply comparing the percentage of best responses is not enough. Instead, we will solve each benchmark with each model several times, each time allowing a different number of generated tokens. We expect that models with higher reasoning skills will improve the most relative to their previous performances when given a higher generating limit. Thus, we provide two metrics to test success. One is performance, which is simply the highest accuracy score obtained by the model at any generating-token limit. The second is scaling, which is defined as the slope of

the curve of generating-tokens vs accuracy. These two metrics will give a more comprehensive analysis of the reasoning capabilities of the model before and after fine-tuning.

2.3. Crosscoder Training and Analysis

The specific goal of the Crosscoder is to compare the internal activations of two related transformer models, the base model and a fine-tuned version of it. The Crosscoder takes activation vectors from a specific, corresponding layer (using PyTorch hooks) from both models simultaneously as input.

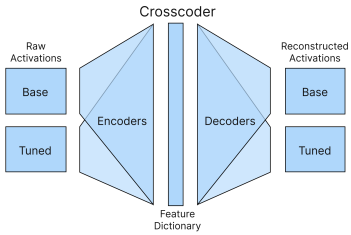


Figure 1. Crosscoder Architecture

The Crosscoder architecture (Fig. 1), based on recent work [14, 1, 12, 3], uses an encoder to map these paired activations into a shared, sparse latent space (16x the models' 1024-dim residual stream). Separate decoders then attempt to reconstruct the original activations for each model from this shared representation, acting as a bottleneck to identify common and distinct features. The total parameter count of the encoders and decoders is $\approx$ 67M. Our implementation extended prior work by Connor Kissane and Neel Nanda [12, 3]. We adapted the loss function and performing hyperparameter tuning specifically for model comparison. Activations for training were generated by running both models (wrapped via TransformerLens [18]) on a 500k sample subset of the Pile dataset [10, 2].

To train the Crosscoder, we used the loss function described in Anthropic's February 2025 update [1].

$$\mathcal{L}_{total} = \mathcal{L}_{L2} + \lambda \mathcal{L}_{L1}$$

The first portion $\mathcal{L}_{L2}$ is the reconstruction loss $\mathcal{L}_{L2} = \sum_m \|x^m - \hat{x}^m\|^2$. Where models $m \in \{\text{base}, \text{tuned}\}$ are used. For each model, x^m is the activation from that model, and $\hat{x}^m$ is the reconstruction using that model's decoder weights. The reconstruction loss encourages the shared latent feature representation $f(x)$ to be useful in reconstructing both sets of activations.

The second portion $\mathcal{L}_{L1}$ is a sparsity penalty that penalizes large activations and encourages sparse usage of features, with L1 norm on decoder weights weighted by the activations $\lambda \mathcal{L}_{L1} = \lambda \sum_i f_i(x) \sum_m \|W^m_{\text{dec},i}\|$. Where λ is a hyperparameter controlling the sparsity regularization strength. Each feature i contributes a sparsity penalty proportional to its activation $f_i(x)$ (i.e., activation-weighted regularization). Both decoders $W^A_{\text{dec},i}$ and $W^B_{\text{dec},i}$ get penalized. This encourages features to be sparse and disentangled, i.e., only a few features are active per input, and those features ideally correspond to differences between models.

Putting it together, the total loss for the crosscoder is

$$\mathcal{L} = \mathbb{E}_x \left[\sum_m \|x^m - \hat{x}^m\|^2 + \lambda \sum_i f_i(x) \sum_m \|W^m_{\text{dec},i}\| \right]$$

The training objective is to minimize a composite loss function that balances reconstruction accuracy and feature sparsity. The contribution of the L1 penalty λ is controlled by an `l1_coeff` hyperparameter, is linearly increased from zero to its target value during the initial phase of training to stabilize learning. The model is trained using the Adam optimizer with standard beta values (beta1=0.9, beta2=0.999) and gradient clipping (max norm 1.0). A learning rate schedule with a constant rate for 80% of the planned training steps before linearly decaying to zero.

PyTorch and TransformerLens [18] were used for the model and for hooking the activations. Weights & Biases is integrated for experiment tracking and logging metrics such as L2 loss, L1 loss, L0 sparsity (average number of non-zero features), and explained variance for both models. Key hyperparameters influencing the training include the learning rate (lr), L1 coefficient (`l1_coeff`), the size of the hidden dictionary (`dict_size`), batch size, and the specific transformer layer chosen (`hook_point`). These are typically determined through a combination of best practices from related work on sparse autoencoders and empirical tuning based on logged results. While not explicitly detailed in the provided code, evaluating generalization would involve testing the trained Crosscoder on activations generated from a held-out portion of the text dataset and analyzing the interpretability and consistency of the learned features. This approach allows for a detailed comparison of how fine-tuning alters the internal computational pathways of a transformer model.

3. Experiments and Results

3.1. Fine Tuning

The final model selected was Qwen2.5-0.5B-Instruct. The model generalized well, without overfitting. The main changes made were tuning the training parameters, specifically block size, per device train batch size, gradient accumulation steps, and gradient checkpointing. The hyperparameters of learning rate was set to 1e-5 and weight decay to 1e-4. The model was trained over 5 epochs and had a warmup ratio of 0.05. A cosine learning rate scheduler was used. ADAM was used as the optimizer with beta1 set to 0.9 and beta2 set to 0.95. The hyperparameters were mainly kept similar to the original paper since

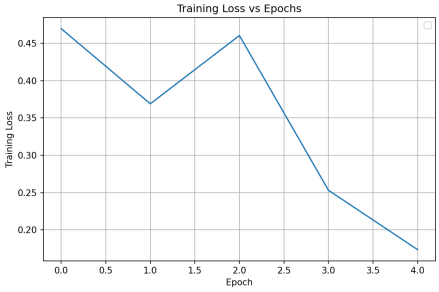


Figure 2. Training loss vs epoch for Qwen model.

the main challenge for training was being able to run it locally and those paramaters yielded good results. The framework used was pytorch, transformers, and trl for superfinetuning. The models and dataset were acquired from huggingface. The llama model did not produce significant results after training. Training was unstable and inconsistent, leading to many crashes. The loss was reduced, but only one epoch of training was possible due to consistent out-of-memory crashes, even with a reduced block size. Also, due to changes in the tokenizer, there may have been mismatches in the model truly learning from the data set. However, the Qwen model performed perfectly without changes in the token structures and was much more stable. This could be due to using the instruct model and the reduction in the number of parameters. Also, since the s1 paper was written using the Qwen2.5-32B-Instruct model, it reasons that running the same model would have better results.

The biggest parameter that affected the training was block size. Originally, the block size was set to 32768 but on local hardware, this was limited to after many different runs to 2048. This number offered the best balance of manageable training time and use of resources while also obtaining great improvement based on the calculated loss. Per device training batch size was set to 2, gradient accumulation steps were set to 4, and gradient checkpointing was enabled. This all was balanced in order to maximize use of the 16 GB VRAM and while maintaining a fast training cycle. Five epochs were selected because more than that showed insignificant improvements on loss. The final training loss achieved was 0.38 after 5 epochs with a runtime of about 30 minutes, similar to what the s1 paper achieved with their cluster of GPUs. Both trained models were then uploaded to their own respective huggingface repository.

3.2. Evaluation Metrics

In [16], AIME24, MATH500, and GPQA Diamond benchmark tests were used. These are all very tough benchmark tasks, related to PhD-level questions on various topics. Although these are good benchmarks for complex models, we found that for our smaller 0.5B parameter model, AIME24 and GPQA were too complex to solve, achieving very close to 0% on both pretrained and finetuned models. For this reason, we replaced these two with simpler, but still reasoning-dependent tasks. The final benchmark tests used were the original Math-500, gsm8k (a task involving grade school level math questions) and boolq (a list of common sense questions).

As expected, from figure 3 we can see that as token length grows, time complexity grows in a $O(n^2)$. This is because, on one hand, more tokens mean more generation tasks. Additionally, each subsequent task gets as input a longer chain of text. Independent from the time taken to execute, we can see from figure 4 that accuracy actually improves for the finetuned model. Table 1 shows the results of both models. As we can see on the table, the finetuned model has a significantly higher scaling on all benchmarks, confirming the reasoning capacities of this model. As for performance, fie-tuned model performed better on both gsm8k and boolq, although there was a slight decrease on boolq. Although the decrease is minor in comparison to the improvement in other benchmarks, it is somewhat concerning. Perhaps it could be explained by a mismatch on the formatting of the questions, or a focus on math questions on the training dataset, rather than general common sense. Regardless of this slight decrement, it is safe to assume that the finetuned model generally performs better on reasoning tasks.

Dataset	Model	Scaling	Performance
gsm8k	Pretrained	-0.0014	41.8
gsm8k	Fine-tuned	0.0277	51.6
boolq	Pretrained	0.0020	11.7
boolq	Fine-tuned	0.0078	10.5
math-500	Pretrained	0.0115	34.8
math-500	Fine-tuned	0.0253	48.0

Table 1. Model Performance Across Datasets

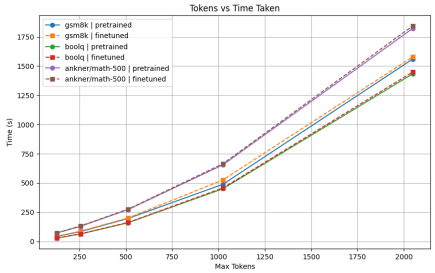


Figure 3. Time taken on running 3 different benchmark tests for pretrained and finetuned models with Nvidia T4 GPU .

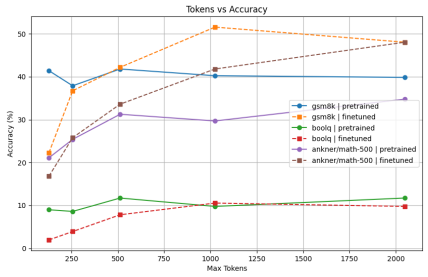


Figure 4. Accuracy of pretrained and finetuned models for 3 different benchmark tests.

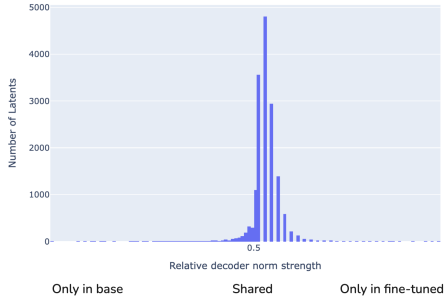


Figure 5. Relative Decoder Norm Strength

Metric	Base	Fine-tuned
Cross-Entropy Loss (Clean)	1.7422	1.8047
Cross-Entropy Loss (Zero Abl.)	12.4375	12.4375
Cross-Entropy Loss (Spliced)	2.5781	2.5781
CE Recovered	0.9219	0.9258

Table 2. Cross-entropy metrics for base and fine-tuned models

3.3. Crosscoder Analysis

3.3.1 Cross-Entropy Loss Recovery Evaluation

To assess if the Crosscoder captures functionally important information at the hooked layer (blocks.14.hook_resid_pre), we measured Cross-Entropy (CE) Loss Recovery. This metric evaluates the impact on the model's primary task (next-token prediction) when substituting original activations with the Crosscoder's reconstructions. We compared the model's standard CE loss ('Clean Loss') against two conditions: one where the layer's activations are replaced entirely by zeros ('Zero-Ablation Loss', representing maximum information loss at that point), and one where they are replaced by the Crosscoder's reconstructions ('Spliced Loss'). High CE Recovery indicates the reconstruction preserves the information necessary for the downstream task. It is calculated as the fraction of the potential performance degradation (the loss increase from Clean to Zero-Ablation) that is prevented by using the Spliced reconstructions. As shown in Table 2, the Crosscoder achieved high recovery for both the base ($\geq$ 92%) and fine-tuned ($\geq$ 93%) models, confirming it effectively represents the functionally relevant aspects of the original activations at this layer, crucial for meaningful subsequent analysis.

3.3.2 Relative Norm Analysis

The first experiment replicates a key finding from Anthropic's work on crosscoders [14] by analyzing feature specificity. It calculates the L2 norm of the decoder weight vector associated with each latent feature, separately for its contribution to reconstructing the base model's activations and the fine-tuned model's activations. By computing the relative norm strength (Tuned norm / (Base norm + Tuned norm)) for each feature and plotting a histogram (Fig. 5), the experiment visually demonstrates that the activations in both models are nearly the same. This is despite the fine-tuned model's significantly improved performance on verifiable domains.

3.3.3 Cosine Similarity Analysis

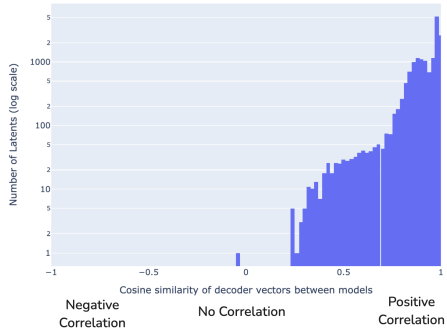


Figure 6. Cosine Similarity of Decoder Vectors

A second experiment investigates the alignment of features deemed "shared" in the previous step (those with relative norms between 0.3 and 0.7). For these shared features, the cosine similarity is calculated between the base model's decoder vector and the fine-tuned model's decoder vector. A histogram (Fig. 6) of these similarities reveals that the vast majority of shared features exhibit very high cosine similarity (close to 1), indicating that they represent similar computational directions in both the base and fine-tuned models, which again corroborates Anthropic's results and suggests that the Crosscoder successfully identifies conserved properties.

4. Anticipated / Encountered Problems

Given the nature of the project, bridging state-of-the-art reasoning fine-tuning with mechanistic interpretability tools on limited hardware, we anticipated several challenges. Firstly, we expected significant computational re-

source constraints, as the original s1 paper [16] utilized substantial GPU clusters (16 H100s) for fine-tuning a 32B parameter model. Secondly, we anticipated that fine-tuning a much smaller model (0.5B parameters) for complex reasoning might yield limited performance improvements or prove difficult to stabilize. Finally, interpreting the features derived from the Crosscoder was expected to be non-trivial, requiring careful analysis to connect latent representations to functional changes in reasoning.

Attempting to reproduce the original setup directly was infeasible. This required a significant downscaling to the Qwen2.5-0.5B-Instruct model and adjustments to training parameters like block size (reduced from 32768 to 2048) for the fine-tuning process. The very first model we attempted to fine-tune, Llama-3.2-1B, did not work well. We encountered stability issues during training, leading to frequent out-of-memory errors and crashes even with reduced block sizes, preventing us from completing more than one epoch effectively. We also suspected potential mismatches with its tokenizer on the s1K dataset format. This initial failure prompted the subsequent pivot to the Qwen2.5-0.5B-Instruct model, which proved much more stable and receptive to the fine-tuning

Another encountered problem stemmed from the model downscaling: the benchmark tests used in the original s1 paper (AIME24, GPQA Diamond) proved far too difficult for our 0.5B parameter model, yielding near-zero accuracy. This required us to adapt our evaluation strategy, substituting these benchmarks with more tractable reasoning tasks like gsm8k and boolq, alongside MATH500, to observe meaningful performance changes. While Crosscoder training was successful after hyperparameter tuning, ensuring the correct implementation of activation scaling for the CE Loss Recovery experiment required careful verification and debugging.

Somewhat naively, we encountered more issues than we anticipated trying to decipher interpretable features from the Crosscoder. We were able to achieve a novel result, but despite significant effort, made less progress unlocking what makes reasoning models work than we had hoped for.

5. Conclusion

This project aimed to investigate the internal mechanistic changes induced when fine-tuning a language model for enhanced step-by-step reasoning. We successfully reproduced key aspects of the s1 fine-tuning methodology [16] on a significantly smaller model, Qwen2.5-0.5B-Instruct, using the s1K dataset. Our evaluation confirmed that this fine-tuning improved the model's reasoning capabilities, particularly its ability to scale performance with increased generation length on mathematical benchmarks (gsm8k, math-500), although a slight degradation was observed on a commonsense task (boolq).

To probe the underlying changes, we trained a Crosscoder [1, 14] comparing the activations of the base and fine-tuned models at a specific layer (mid-network residual stream). The Crosscoder achieved high Cross-Entropy Loss recovery ($\geq$93%), indicating it effectively captured the functionally relevant information at that layer. However, subsequent analysis using relative decoder norms and cosine similarity, replicating methods from Anthropic [1], revealed that the vast majority of learned features were shared between the base and fine-tuned models and highly aligned in their representational direction with the chosen layer.

This central finding presents an intriguing picture: despite measurable improvements in reasoning behavior (especially scaling), the fine-tuning process induced remarkably subtle changes in the representational geometry at the analyzed mid-network layer, at least as captured by the Crosscoder. This suggests that the enhanced reasoning capabilities might come from: (a) modifications in layers not analyzed (ones closer to the output) or (b) changes in how existing features are utilized or combined (e.g., via attention, which the Crosscoder doesn't directly model). This underscores the challenge in pinpointing specific, interpretable circuit-level modifications responsible for complex changes in model behavior like reasoning. Nonetheless, the project successfully demonstrates the feasibility of applying advanced interpretability techniques like Crosscoers to study fine-tuning effects and highlights the potential for distilling reasoning capabilities into smaller models.

5.1. Limitations and future work

Our study has several limitations. We focused on a relatively small model (0.5B parameters), and findings may not directly generalize to larger, state-of-the-art models. Our Crosscoder analysis was restricted to a single layer; changes crucial for reasoning might be more prominent elsewhere, particularly in later layers. The Crosscoder itself, while powerful, provides one specific view of representational change and may miss other forms of modification. The slight performance decrease on BoolQ warrants further investigation.

Future work could address these limitations. Analyzing multiple layers simultaneously or focusing on later layers using Crosscoders or related techniques, such as SAEs or circuit analysis, could yield more insights. Investigating the BoolQ performance drop might reveal interesting tradeoffs during reasoning specialization. Finally, applying this methodology to study different reasoning datasets or scaling the analysis to larger models (resource permitting) would be valuable extensions.

6. Contributions

See Table 3.

Student Name	Contributed Aspects	Details
Carlos Iraheta	Base Model Fine Tuning	Chose the models, dataset and performed training locally to improve model's ability to reason
Juan Molina	Evaluation Metrics	Chose, implemented, ran, and analyzed 3 benchmark tests that showcased reasoning capabilities for the model before and after fine-tuning
Tyler Crosse	Crosscoder training and Analysis	Trained the crosscoder on the activations of the base and fine-tuned models when activated on a 500k sample subset of the pile. Analyzed the crosscoder.

Table 3. Contributions of team members.

References

[1] Insights on Crosscoder Model Diffing. 1, 2, 3, 6

[2] Jbrinkma/pile-500k · Datasets at Hugging Face. 3

[3] Neelnanda-io/Crosscoders. 3

[4] Simplescaling/s1K-1.1 · Datasets at Hugging Face. 1, 2

[5] Stage-Wise Model Diffing. 1

[6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020. 1

[7] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. 1, 2

[8] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy Models of Superposition. 1, 2

[9] Andrey Galichin, Alexey Dontsov, Polina Druzhinina, Anton Razzhigaev, Oleg Y. Rogov, Elena Tutubalina, and Ivan Oseledets. I Have Covered All the Bases Here: Interpreting Reasoning Features in Large Language Models via Sparse Autoencoders. 2

[10] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800GB Dataset of Diverse Text for Language Modeling. 3

[11] Jie Huang and Kevin Chen-Chuan Chang. Towards reasoning in large language models: A survey. *arXiv preprint arXiv:2212.10403*, 2022. 1

[12] Connor Kissane, robertzk, Arthur Conmy, and Neel Nanda. Open Source Replication of Anthropic's Crosscoder paper for model-diffing. 3

[13] Dacheng Li, Shiyi Cao, Chengkun Cao, Xiuyu Li, Shangyin Tan, Kurt Keutzer, Jiarong Xing, Joseph E. Gonzalez, and Ion Stoica. S*: Test Time Scaling for Code Generation. 1

[14] Jack Lindsey, Adly Templeton, Jonathan Marcus, Thomas Conerly, Joshua Batson, and Christopher Olah. Sparse Crosscoders for Cross-Layer Features and Model Diffing. 1, 2, 3, 5, 6

[15] Julian Minder, Clement Dumas, Caden Juang, Bilal Chugtai, and Neel Nanda. Robustly identifying concepts introduced

during chat fine-tuning using crosscoders. 1

[16] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. S1: Simple test-time scaling. 1, 2, 4, 6

[17] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. S1: Simple test-time scaling. 2

[18] Neel Nanda and Joseph Bloom. Transformerlens. https://github.com/TransformerLensOrg/TransformerLens, 2022. 3

[19] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. 1

[20] OpenAI. OpenAI o3 and o4-mini System Card. 1, 2

[21] OpenAI, Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, Alex Iftimie, Alex Karpenko, Alex Tachard Passos, Alexander Neitz, Alexander Prokofiev, Alexander Wei, Allison Tam, Ally Bennett, Ananya Kumar, Andre Saraiva, Andrea Vallone, Andrew Duberstein, Andrew Kondrich, Andrey Mishchenko, Andy Applebaum, Angela Jiang, Ashvin Nair, Barret Zoph, Behrooz Ghorbani, Ben Rossen, Benjamin Sokolowsky, Boaz Barak, Bob McGrew, Borys Minaiev, Botao Hao, Bowen Baker, Brandon Houghton, Brandon McKinzie, Brydon Eastman, Camillo Lugaresi, Cary Bassin, Cary Hudson, Chak Ming Li, prefix=de-useprefix=false family=Bourcy, given=Charles, Chelsea Voss, Chen Shen, Chong Zhang, Chris Koch, Chris Orsinger, Christopher Hesse, Claudia Fischer, Clive Chan, Dan Roberts, Daniel Kappler, Daniel Levy, Daniel Selsam, David Dohan, David Farhi, David Mely, David Robinson, Dimitris Tsipras, Doug Li, Dragos Oprica, Eben Freeman, Eddie Zhang, Edmund Wong, Elizabeth Proehl, Enoch Cheung, Eric Mitchell, Eric Wallace, Erik Ritter, Evan Mays, Fan Wang, Felipe Petroski Such, Filippo Raso, Florencia Leoni, Foivos Tsimpourlas, Francis Song, prefix=von-useprefix=false family=Lohmann, given=Fred, Freddie Sulit, Geoff Salmon, Giambattista Parascandolo, Gildas Chabot, Grace Zhao, Greg Brockman, Guillaume Leclerc, Hadi Salman, Haiming Bao, Hao Sheng, Hart Andrin, Hessam Bagherinezhad, Hongyu Ren, Hunter Lightman, Hyung Won Chung, Ian Kivlichan, Ian O'Connell, Ian Osband, Ignasi Clavera Gilaberte, Ilge Akkaya, Ilya Kostrikov, Ilya Sutskever, Irina Kofman, Jakub Pachocki, James Lennon, Jason Wei, Jean Harb, Jerry Twore, Jiacheng Feng, Jiahui Yu, Jiayi Weng, Jie Tang, Jieqi Yu, Joaquin Quiñonero Candela, Joe Palermo, Joel Parish, Johannes Heidecke, John Hallman, John Rizzo, Jonathan Gordon, Jonathan Uesato, Jonathan Ward, Joost Huizinga, Julie Wang, Kai Chen, Kai Xiao, Karan Singhal, Karina Nguyen, Karl Cobbe, Katy Shi, Kayla Wood, Kendra Rimbach, Keren Gu-Lemberg, Kevin Liu, Kevin Lu, Kevin Stone, Kevin Yu, Lama Ahmad, Lauren Yang, Leo Liu, Leon Maksin, Leyton Ho, Liam Fedus, Lilian Weng, Linden Li, Lindsay McCallum, Lindsey Held, Lorenz Kuhn, Lukas Kondraciuk, Lukasz Kaiser, Luke Metz, Madelaine Boyd, Maja Trebacz, Manas Joglekar, Mark Chen, Marko Tintor, Mason Meyer, Matt Jones, Matt Kaufer, Max Schwarzer, Meghan Shah, Mehmet Yatbaz, Melody Y. Guan, Mengyuan Xu, Mengyuan Yan, Mia Glaese, Mianna Chen, Michael Lampe, Michael Malek, Michele Wang, Michelle Fradin, Mike McClay, Mikhail Pavlov, Miles Wang, Mingxuan Wang, Mira Murati, Mo Bavarian, Mostafa Rohaninejad, Nat McAleese, Neil Chowdhury, Neil Chowdhury, Nick Ryder, Nikolas Tezak, Noam Brown, Ofir Nachum, Oleg Boiko, Oleg Murk, Olivia Watkins, Patrick Chao, Paul Ashbourne, Pavel Izmailov, Peter Zhokhov, Rachel Dias, Rahul Arora, Randall Lin, Rapha Gontijo Lopes, Raz Gaon, Reah Miyara, Reimar Leike, Renny Hwang, Rhythm Garg, Robin Brown, Roshan James, Rui Shu, Ryan Cheu, Ryan Greene, Saachi Jain, Sam Altman, Sam Toizer, Sam Toyer, Samuel Miserendino, Sandhini Agarwal, Santiago Hernandez, Sasha Baker, Scott McKinney, Scottie Yan, Shengjia Zhao, Shengli Hu, Shibani Santurkar, Shraman Ray Chaudhuri, Shuyuan Zhang, Siyuan Fu, Spencer Papay, Steph Lin, Suchir Balaji, Suvansh Sanjeev, Szymon Sidor, Tal Broda, Aidan Clark, Tao Wang, Taylor Gordon, Ted Sanders, Tejal Patwardhan, Thibault Sottiaux, Thomas Degry, Thomas Dimson, Tianhao Zheng, Timur Garipov, Tom Stasi, Trapit Bansal, Trevor Creech, Troy Peterson, Tyna Eloundou, Valerie Qi, Vineet Kosaraju, Vinnie Monaco, Vitchyr Pong, Vlad Fomenko, Weiyi Zheng, Wenda Zhou, Wes McCabe, Wojciech Zaremba, Yann Dubois, Yinghai Lu, Yining Chen, Young Cha, Yu Bai, Yuchen He, Yuchen Zhang, Yunyun Wang, Zheng Shao, and Zhuohan Li. OpenAI o1 System Card. 1, 2

[22] Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 Technical Report. 2

[23] Lee Sharkey, Bilal Chughtai, Joshua Batson, Jack Lindsey, Jeff Wu, Lucius Bushnaq, Nicholas Goldowsky-Dill, Stefan Heimersheim, Alejandro Ortega, Joseph Bloom, Stella Biderman, Adria Garriga-Alonso, Arthur Conmy, Neel Nanda, Jessica Rumbelow, Martin Wattenberg, Nandi Schoots, Joseph Miller, Eric J. Michaud, Stephen Casper, Max Tegmark, William Saunders, David Bau, Eric Todd, Atticus Geiger, Mor Geva, Jesse Hoogland, Daniel Murfet, and Tom McGrath. Open Problems in Mechanistic Interpretability. 1, 2

[24] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. 1, 2

[25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszko-

reit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 1

[26] Xinyi Wang, Lucas Caccia, Oleksiy Ostapenko, Xingdi Yuan, William Yang Wang, and Alessandro Sordoni. Guiding Language Model Reasoning with Planning Tokens. 1

[27] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. 1

[28] Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Zheng Lin, Li Cao, and Weiping Wang. Dynamic Early Exit in Reasoning Models. 1